

Cricket Live Score & Statistics Application

Product Requirement & Technical Project Documentation — Version 1.0

Functional & Technical Handover Document

1. Project Overview

The objective is to develop a dedicated Cricket Live Score and Statistics application inspired by the information architecture and live-match experience of platforms such as Cricbuzz and Sofascore. The application will focus exclusively on cricket.

SportsAPI365 Cricket APIs will be used as the primary external cricket data source. The application will provide live scores, upcoming and completed matches, ball-by-ball commentary, scorecards, team information, player information, competition information and cricket search.

2. Project Scope

In Scope: Cricket only; live matches; upcoming matches; completed matches; match details; live score; ball-by-ball commentary; scorecard; overs; innings; match information; H2H where available; teams; players; competitions; standings where available; global cricket search; responsive web interface.

Out of Scope for V1: Other sports offered by SportsAPI365, native mobile applications, user betting/wagering features, and unrelated sports modules.

3. Technology Stack

Layer	Technology
Frontend	Next.js + React.js
Styling	Tailwind CSS
Backend	Python + FastAPI
Cache	Redis
Database	PostgreSQL
External Data Provider	SportsAPI365 Cricket API

Architecture principle: the frontend must never call SportsAPI365 directly. All external API communication must be handled by FastAPI.

4. External Cricket API Scope

The project will use the following SportsAPI365 Cricket API groups: Match Details; Schedules & Lists; Team Lists; Team Profile; Managers; Player Lists; Player Profile; Competition Lists; Competition Profile; Search.

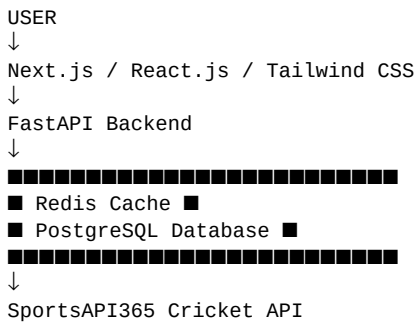
API Group	Purpose
Match Details	Live match data, score, ball-by-ball, innings, overs, H2H, best players and related match information
Schedules & Lists	Today's fixtures, events and match discovery
Team Lists / Profile	Team directory, profile, players, events, performance and competition-related data
Managers	Manager/coach information where available
Player Lists / Profile	Player directory, profile, history and competition statistics
Competition Lists / Profile	Tournament discovery, matches, seasons and standings
Search	Search across cricket players, teams and competitions

Reference documentation: <https://sportsapi365.com/docs/direct/cricket/match-details> ;
<https://sportsapi365.com/docs/direct/cricket/schedules-lists> ; <https://sportsapi365.com/docs/direct/cricket/team-lists> ;
<https://sportsapi365.com/docs/direct/cricket/team-profile> ; <https://sportsapi365.com/docs/direct/cricket/managers> ;
<https://sportsapi365.com/docs/direct/cricket/player-lists> ; <https://sportsapi365.com/docs/direct/cricket/player-profile> ;
<https://sportsapi365.com/docs/direct/cricket/competition-lists> ; <https://sportsapi365.com/docs/direct/cricket/competition-profile> ;
<https://sportsapi365.com/docs/direct/cricket/search>

5. High-Level System Architecture

The recommended architecture is: User → Next.js/React → FastAPI → Redis/PostgreSQL → SportsAPI365.

SportsAPI365 should be treated as an external source of truth for cricket event data, while Redis provides fast-access cache and PostgreSQL provides persistent normalized storage for application use.



6. API Rate Limit & Request Control

The specified external API limit is 1 request every 2 seconds. This must be treated as a global backend constraint.

The frontend must not poll SportsAPI365. Multiple users viewing the same match must receive data from the application's cache rather than generating separate external API requests.

WRONG:
500 users → 500 external API requests

CORRECT:
500 users → FastAPI → Redis cached data
↓
controlled API refresh

A centralized request queue/rate limiter should ensure that outbound SportsAPI365 requests are serialized or otherwise controlled so the configured minimum interval is respected. The implementation should make the interval configurable.

7. Application Modules

The application should contain the following primary modules: Home, Live Matches, Upcoming Matches, Recent Matches, Match Details, Teams, Players, Competitions and Search.

```
HOME
■■■ Live Matches
■■■ Upcoming Matches
■■■ Recent / Completed Matches
■■■ Match Details
■ ■■■ Summary
■ ■■■ Commentary
■ ■■■ Scorecard
■ ■■■ Overs
■ ■■■ Info
■ ■■■ H2H
■■■ Teams
■■■ Players
■■■ Competitions
■■■ Search
```

8. Development Phase 1 — Project Setup

Set up the Next.js frontend, React component architecture, Tailwind CSS, FastAPI backend, Redis and PostgreSQL. Configure environment variables, API credentials, logging, error handling and separate development/staging/production configuration.

Frontend structure should use reusable components and a dedicated service layer. Backend should use a service/repository pattern so external API access, caching and database persistence remain separated.

15. Scorecard

The scorecard should show innings-wise batting and bowling information based on the fields actually returned by the provider.

```
BATSMAN R B 4s 6s SR
Player A 72 45 8 2 160.0
Player B 41 32 4 1 128.1
```

```
BOWLER O M R W ECO
Bowler A 4 0 32 2 8.00
```

16. Overs & Innings

The Overs section should provide over-by-over run breakdown and delivery details where available. The Innings section should provide innings number, batting team, total, wickets, overs and progression.

Do not hard-code cricket statistics that are not present in the provider response. UI fields should be driven by the normalized API model.

17. H2H & Match Enrichment

Where the relevant SportsAPI365 endpoints return data, provide H2H, best players, pregame form, team streaks and other match enrichment as additional sections.

These features should be implemented after the core live-score, commentary and scorecard flow is stable.

18. Team Module

Team List route: /teams. Team Profile route: /team/{teamId}.

Team listing should support search/filter where available. Team profile should contain overview, players, upcoming matches, recent matches, performance, rankings and competitions where corresponding data is available.

19. Player Module

Player List route: /players. Player Profile route: /player/{playerId}.

Player profile should show identity information, current team where available, match history and tournament/season statistics supported by the API.

20. Competition Module

Competition List route: /competitions. Competition Profile route: /competition/{competitionId}.

Competition profile should contain competition header, current season, fixtures, completed matches, standings and past seasons where available.

21. Search Module

Provide a global cricket search in the header. Results should be grouped into Players, Teams and Competitions. Debouncing should be implemented on the frontend to avoid unnecessary backend requests.

22. Backend API Structure

```
GET /api/v1/matches/live
GET /api/v1/matches/upcoming
GET /api/v1/matches/recent
GET /api/v1/matches/{eventId}
GET /api/v1/matches/{eventId}/commentary
GET /api/v1/matches/{eventId}/scorecard
GET /api/v1/matches/{eventId}/overs
GET /api/v1/matches/{eventId}/h2h
```

```
GET /api/v1/teams
GET /api/v1/teams/{teamId}
GET /api/v1/players
GET /api/v1/players/{playerId}
GET /api/v1/competitions
GET /api/v1/competitions/{competitionId}
GET /api/v1/search
```

These are internal application APIs. The exact response contracts should be defined during backend implementation and versioned under /api/v1.

23. Data Synchronization Strategy

Live matches should receive the highest refresh priority. Upcoming fixtures, teams, players and competitions can use lower refresh frequencies. Once a match is completed, the system should persist the final state and stop aggressive live refresh.

```
LIVE MATCH
↓
Controlled Scheduler
↓
SportsAPI365
↓
Normalize Response
↓
Redis
↓
PostgreSQL where required
↓
FastAPI
↓
Frontend
```

24. Cache Strategy

Data Type	Priority	Typical Handling
Live score	Very High	Frequent controlled refresh + Redis
Ball-by-ball	Very High	Frequent controlled refresh + Redis
Today's fixtures	High	Scheduled refresh + Redis
Upcoming matches	Medium	Periodic refresh
Team profile	Low/Medium	Cache and refresh periodically
Player profile	Low/Medium	Cache and refresh periodically
Competition profile	Low/Medium	Cache and refresh periodically
Historical data	Low	Persist and reuse

TTL and refresh intervals must remain configurable because the external provider's limits and product requirements may change.

25. Error Handling

If SportsAPI365 is temporarily unavailable, the backend should first check whether usable cached data exists. If available, return the latest cached data with an appropriate freshness indicator. If not available, return a structured error response.

Frontend must provide proper loading, empty and error states rather than showing a blank page.

26. Security

SportsAPI365 API credentials must be stored in server-side environment variables or a secure secrets mechanism. API keys must never be exposed to the browser.

Validate all incoming IDs and query parameters. Implement backend logging without exposing API keys or sensitive credentials.

27. Responsive UI Requirements

The application must be responsive for desktop, laptop, tablet and mobile. Mobile and desktop layouts should both prioritize quick access to live scores and commentary.

Use reusable cards, tabs, tables, badges, skeletons and empty/error components across modules.

28. UI/UX Direction

The product may take inspiration from the information density and usability patterns of Cricbuzz/Sofascore, but the visual design, component implementation and branding should be independently created.

Primary UX goals: fast score visibility, clear team hierarchy, compact match cards, easy commentary access, minimal navigation friction and consistent responsive behavior.

29. V1 Feature Priorities

Priority	Features
P0 — Core	Home, live matches, upcoming matches, recent matches, match details, live score, commentary, scorecard, overs, innings, match info
P1 — Data Modules	Teams, team profiles, players, player profiles, competitions, competition profiles, standings where available
P2 — Enrichment	H2H, best players, form, streaks and additional statistics where available

30. Recommended Development Order

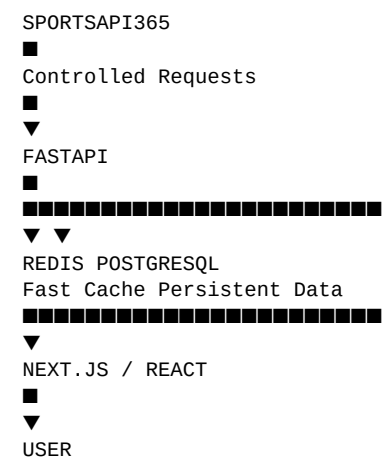
1. Next.js + Tailwind setup
2. FastAPI setup
3. SportsAPI365 configuration
4. API client/service layer
5. Global rate-limit controller
6. Redis setup
7. PostgreSQL schema
8. Match synchronization service
9. Home page
10. Live match cards
11. Upcoming/recent cards
12. Match Details page
13. Live score
14. Ball-by-ball commentary
15. Scorecard
16. Overs
17. Innings
18. H2H / Best Players / Form
19. Team module
20. Player module
21. Competition module
22. Search
23. Responsive optimization
24. Cache/API optimization
25. Testing
26. Production deployment

31. Acceptance Checklist

Area	Acceptance Criteria
API	All required cricket API groups are integrated through FastAPI
Rate Limit	Outbound provider calls are centrally controlled and respect the configured 2-second interval
Security	Provider API key is never exposed to frontend
Home	Live, upcoming and completed match cards are displayed

Area	Acceptance Criteria
Match	Match details load correctly by event ID
Live	Live score updates through backend-controlled synchronization
Commentary	Ball-by-ball data is displayed in readable chronological order
Scorecard	Batting/bowling/innings data is rendered from normalized API response
Teams	Team listing and profiles work
Players	Player listing and profiles work
Competitions	Competition listing and profiles work
Search	Players, teams and competitions can be searched
Cache	Redis is used for high-frequency data
Database	Persistent data and synchronization metadata are stored in PostgreSQL
UI	Responsive on desktop and mobile
Errors	Loading, empty, stale and API error states are handled

32. Final Architecture Summary



The core principle is to build the product around a controlled backend data layer rather than exposing or repeatedly calling the external provider from the browser. This is essential for respecting the external request limit and for supporting multiple concurrent users.